

Take-Home Exercise

Project Name: TaskHub

You are building a small but realistic system that a consultancy could hand over to a client. This exercise evaluates you across the full SDLC: planning, analysis, design, implementation, testing/integration, and maintenance/operations.

You will deliver one codebase. Focus on correctness, security, maintainability, testability, and handover-quality documentation.

Tech stack

- Frontend: React + TypeScript
- Backend: .NET (Core) + C#
- Storage: must support both In-Memory and File-Based storage, switchable via configuration
- You may use additional libraries, but you must justify each dependency in a dependency register

Security and API reference points (you are expected to align with these in spirit and cite them in your research log)

- OWASP ASVS (latest major release is v5.x)
- OWASP Top 10 (latest released version is 2025)
- Cookie + SameSite considerations for ASP.NET Core
- Conditional requests / optimistic concurrency using ETags + If-Match (HTTP Semantics RFC 9110)
- Problem Details error format (RFC 7807 / RFC 9457)

Submission format

- Zip the repository (or provide a private repo link)
- Send to: careers@elios.tech
- Your repo must be runnable on a fresh machine with only standard toolchains installed (Node + .NET SDK)

Important note about sources

- If you cite sources in your research log, include a short note on how the source influenced a concrete decision in your solution.

1) What we are testing

We are explicitly testing that you can:

- Plan and manage scope like an engineer working with real clients
 - Translate ambiguous needs into clear requirements and acceptance criteria
 - Produce a design that is secure, maintainable, testable, and operable
 - Implement cleanly with correct boundaries and good engineering standards
 - Prove correctness with tests at multiple levels + automation
 - Handle maintenance tasks: bugfix, change request, and security hardening
 - Produce handover-quality documentation
-

2) Product scope

TaskHub is a multi-tenant to-do platform:

- Users belong to one or more Organisations (Workspaces/Tenants)
 - All todos are scoped to an Organisation
 - Role-based access control (RBAC): Member vs OrgAdmin
 - Auditing of important events
 - Import/export to support onboarding/offboarding and client handover
 - Concurrency safety to prevent lost updates
 - Two storage implementations: InMemory and File store with schema versioning + migration
-

3) Functional Requirements

3.1 Identity, authentication, authorisation (RBAC)

You must implement:

- Register user (minimal PII; use username or email + password)
- Login and logout
- Session-based authentication using cookies
- Password hashing using a modern password hashing algorithm (bcrypt/Argon2)
- Protection against brute force:
 - either rate limiting, throttling, or lockout (document and test)

- Prevent user enumeration:
 - auth failures must not reveal whether the username exists
- RBAC:
 - `Member` : can manage their own todos inside an organisation they belong to
 - `OrgAdmin` : can manage members of that organisation and can view audit logs for that organisation

Security expectations (we will look for these)

- Cookie configuration should be intentional (HttpOnly, Secure policy, SameSite policy explained)
- CSRF strategy for state-changing requests must be explicit and justified

3.2 Multi-tenancy and membership

You must implement:

- Create organisation
- Invite/add user to organisation (your choice of mechanism; keep simple but robust)
- Switch active organisation in the UI
- Enforcement on the API:
 - every todo operation must be scoped to an organisation
 - a user must not access data from an organisation they're not a member of
 - only OrgAdmins can manage membership and view audit logs

3.3 Todo domain

Todo fields (minimum)

- `id`
- `orgId`
- `title` (required, non-empty, length limits)
- `description` (optional, length limits)
- `status` (Open | Done)
- `priority` (Low | Medium | High)
- `tags` (0..n, validate allowed characters/length)
- `dueDate` (optional)

- `createdAt` , `updatedAt`
- `version` (for optimistic concurrency)

Lifecycle requirements

- Create todo
- List todos with:
 - pagination (required)
 - sorting (`createdAt`, `dueDate`, `priority`)
 - filtering (`status`, `overdue`, `tag`)
- Update todo (must support partial updates or full updates; your choice, justify)
- Toggle status (Open <-> Done)
- Soft delete todo (recoverable)
- Restore soft-deleted todo
- Hard delete todo (OrgAdmin only)

UI requirements (functional, not pretty)

- Login/Registration screens
- Organisation selection / switching
- Todo list page with:
 - filters, sorting, pagination controls
 - create/edit form
 - optimistic toggle for status with rollback on failure
 - clear loading and error states
- Basic accessibility:
 - keyboard-usable main flows
 - labels for inputs and buttons
 - no “mystery” icon buttons

3.4 Concurrency Control

You **MUST** implement protection against lost updates:

- Use optimistic concurrency with `version` or HTTP ETag

- If using ETag/If-Match:
 - return ETag for todo resources
 - require If-Match on update/delete/restore
 - return 412 Precondition Failed (or 409 Conflict if you justify it) on mismatch
- Provide at least 1 automated test proving the conflict behaviour

3.5 Audit Logging

Record audit events for:

- Auth: login success, login failure, logout
- Todo: create, update, soft delete, restore, hard delete
- Org: create org, membership added/removed, role changes

Each audit entry must include:

- timestamp
- actor userId
- orgId
- action type
- entity type + entityId
- correlationId (trace request across systems)

Audit log viewing

- OrgAdmin can view audit entries for their organisation
- Members cannot view audit logs

3.6 Import/Export (with validation + reporting)

Export

- Export todos for an organisation as JSON or CSV (your choice)
- Must include enough fields to re-import (title, description, status, tags, dueDate, priority)

Import

- Import the same format
- Validate each record and produce an import report:
 - accepted count
 - rejected count

- list of rejected rows with reasons (do not leak sensitive info)
 - Import must be idempotent or have a documented idempotency strategy:
 - example: allow `clientProvidedId` and de-duplicate, or require an idempotency key on import request
-

4) Storage Requirements

You must implement storage abstraction with configuration (both **MUST** be implemented):

- `StorageProvider=InMemory`
- `StorageProvider=File`

4.1 InMemory

- Still must respect concurrency behaviour (version/ETag must work)

4.2 File Storage

You must implement (robust implementation):

- Atomic writes (no partial writes/corruption on crash)
- Concurrency safety (avoid two requests corrupting the file)
- Schema Versioning:
 - persist `schemaVersion` in the file
 - provide a migration from v1 → v2 (can be simple but must be real)
 - provide a test proving migration works

File storage scope

- It's acceptable to store a single JSON file per org, or a single file for everything, as long as it's safe and documented
 - If you choose one-file-per-org, document directory layout and locking strategy
-

5) API Requirements

5.1 Versioning and conventions

- All endpoints under `/api/v1`
- Consistent error format
- Consistent pagination format (document your chosen convention)
- Consistent filtering/sorting conventions (document them)

5.2 Error Format

- Use Problem Details style errors (RFC 7807 or RFC 9457) consistently
- Requirements for errors:
 - machine-readable code/type
 - human-readable title
 - correlationId included
 - validation errors must return field-level details
 - never expose stack traces or internals

5.3 OpenAPI

- Provide OpenAPI/Swagger output from the backend
 - Include at least one example request/response for every endpoint
 - Your frontend must not “hand-roll” types:
 - either generate a typed client from OpenAPI, or define a shared contract module and prove it stays in sync (document approach)
-

6) Non-Functional Requirements

6.1 Observability and diagnosability

Required

- Structured logging (not plain concatenated strings)
- Correlation ID end-to-end:

- frontend generates or requests a correlationId and sends it with every request
- backend logs it and includes it in error responses
- Health endpoints:
 - liveness: “process is up”
 - readiness: “can serve requests” (e.g., storage load, migration status)

6.2 Security Baseline

Required

- Threat model with mitigations mapped
- At minimum address risks aligned with OWASP Top 10
- At minimum map your implemented controls to a small subset of OWASP ASVS checks

6.3 Accessibility Baseline

Required

- Keyboard navigation for core flows
- Labelled inputs
- Visible error messages associated with fields

6.4 Maintainability Baseline

Required

- Clear layering boundaries (domain/application/infrastructure or equivalent)
- Clear module ownership (where things live and why)
- Minimal coupling between UI components and API transport details

7) SDLC Deliverables

Your repository must include:

- `/docs` for planning/analysis/design/testing/ops documentation
- `/maintenance` for maintenance simulations (bugfix/change/security)
- `README.md` and `CHANGELOG.md`

7.1 Planning Deliverables

1. Project Charter

- goals and success criteria
- scope boundaries (MVP vs later)
- constraints (tech, time, risk)
- definition of done
- stakeholder assumptions (who cares about what)

2. Backlog + Prioritisation

- at least 30 backlog items
- each item has acceptance criteria
- identify dependencies and sequencing
- include a release plan (MVP release + next release)

3. Estimation Approach

- choose a simple estimation method (T-shirt sizes, points, etc.)
- explain how you'd use it in a team

4. Risk Register

- at least 10 risks
- include security, delivery, technical, operational, product risks
- each risk has likelihood, impact, mitigation, and early warning signals

5. Delivery Plan

- what you would do week-by-week for a real client
- include checkpoints: design review, security review, pre-release review

Required filenames

- `/docs/01-project-charter.md`
- `/docs/02-backlog.md`
- `/docs/03-estimation-and-plan.md`
- `/docs/04-risk-register.md`

7.2 Analysis Deliverables

1. Requirements specification

- at least 3 personas (include OrgAdmin)
- user journeys:
 - happy paths for Member and OrgAdmin
 - at least 8 failure paths (auth failures, forbidden access, validation, concurrency conflict, import failures)
- acceptance criteria for every functional area (auth, orgs, todos, audit, import/export)

2. Data classification + privacy note

- what data you store
- what is PII
- retention expectations (even if simple)
- what you intentionally do not store/log

3. Research log (required)

- a list of at least 10 sources you consulted
- for each: what decision it influenced
- include security and protocol references (OWASP ASVS, OWASP Top 10, cookie guidance, concurrency/ETag, Problem Details)

Required filenames

- `/docs/05-requirements.md`
- `/docs/06-data-and-privacy.md`
- `/docs/07-research-log.md`

7.3 Design Deliverables

1. Architecture pack (C4)

- Context diagram
- Container diagram
- Component diagram (backend at minimum; frontend optional)
- explain storage provider switching and key boundaries

2. API contract document

- endpoints list
- request/response schemas
- pagination/filtering/sorting conventions
- auth model and session lifecycle

- example request/response per endpoint

3. Data model + invariants

- entity relationships (User, Org, Membership, Todo, AuditEntry)
- invariants (org scoping, role checks, soft delete rules)
- state machine for Todo lifecycle
- concurrency design (version/ETag approach)

4. Threat Model

- at least 15 threats
- use STRIDE or another method (state which)
- map each threat to mitigations (implemented vs deferred)
- call out top residual risks and what you'd do next

5. Operational Design

- logging fields (include correlationId)
- health checks
- storage migration strategy
- backup/restore strategy for file storage (even if basic)

Required filenames

- `/docs/08-architecture.md`
- `/docs/09-api-contract.md`
- `/docs/10-data-model-and-state.md`
- `/docs/11-threat-model.md`
- `/docs/12-ops-design.md`

7.4 Implementation Deliverables

Required in repo

- `.editorconfig` (or equivalent) with formatting rules
- linting/formatting setup for frontend
- formatting/static analysis setup for backend (or document what you used)
- dependency register:
 - `/docs/13-dependency-register.md` listing each library, why it exists, and what risk it introduces

7.5 Testing and Integration Deliverables

1. Test strategy

- unit vs integration vs e2e rationale
- what is covered and what is intentionally not covered
- how tests are structured and named
- test data strategy

2. Test coverage expectations (minimum required)

Backend

- unit tests:
 - permissions checks (Member vs Admin)
 - validation rules
 - concurrency conflict detection
 - import validation and rejection reporting
- integration tests:
 - auth flow + session behaviour
 - multi-tenant enforcement (cannot access other org)
 - audit log written correctly
 - soft delete + restore + hard delete authorization
 - file storage migration test (v1 → v2)

Frontend

- component tests or contract tests:
 - login/register form validation
 - todo list rendering and error states
 - optimistic toggle rollback on conflictE2E (required, automated)
- Flow A (Member):
 - register/login → create org → create todos → filter/sort/paginate → export
- Flow B (OrgAdmin):
 - add member → change role → view audit log → soft delete + restore → import with rejection report

3. CI pipeline (required)

- build backend + frontend
- run all tests
- lint/format checks
- produce artifacts (test report output or coverage summary)

Required filenames

- `/docs/14-test-strategy.md`
 - plus CI config in repo (GitHub Actions is fine)
-

8) Maintenance Simulation

You must complete all three categories below, including triage notes, implementation, tests, and doc updates.

8.1 Bugfix ticket

Ticket title

- "Rapid toggling of todo status sometimes shows incorrect UI state until refresh."

Requirements

- Write a reproduction note:
 - steps to reproduce
 - observed vs expected
- Identify likely root cause(s)
- Implement a fix
- Add automated coverage:
 - at least one test proving it's fixed (component/e2e is ideal)
- Add a short "why it happened" section

Required files

- `/maintenance/01-bug-repro-and-fix.md`

8.2 Change Request (must implement)

Ticket title

- “Archive completed todos older than N days.”

Requirements

- Implement as a background job (in-process is acceptable)
- Provide configuration:
 - `ARCHIVE_AFTER_DAYS`
 - job schedule/interval
- Define what “archive” means:
 - archived items do not show in default list, but can be filtered to show
 - archived items remain restorable (document rules)
- Add tests for archiving behaviour
- Update docs and API contract if needed

Required files

- `/maintenance/02-archive-change.md`

8.3 Security hardening task (must implement)

Ticket title

- “Protect authentication endpoints from scripted abuse without harming UX.”

Requirements

- Choose and implement one:
 - rate limiting
 - progressive backoff
 - temporary lockout
- Must be configurable
- Must be tested (integration test minimum)

- Must be documented (policy + tradeoffs)

Required files

- `/maintenance/03-auth-abuse-protection.md`

8.4 Release Notes & Operational Updates

You must include:

- `CHANGELOG.md` with entries for:
 - bugfix
 - archive feature
 - auth abuse protection
- A short “release note” aimed at a non-technical stakeholder
- A post-incident write-up (simulated) for one incident:
 - file storage corruption discovered, or
 - audit log missing entries for a period, or
 - import caused performance degradation

Required files

- `/maintenance/04-release-notes.md`
- `/maintenance/05-post-incident-report.md`
- `/CHANGELOG.md`

9) Repository Structure

Required

- `/frontend`
- `/backend`
- `/docs`
- `/docs/adr`
- `/maintenance`
- `README.md`
- `CHANGELOG.md`

10) README Requirements

Your `README.md` must include:

- What this system is and what problem it solves
- How to run locally:
 - in-memory storage
 - file storage
- How to run tests (unit, integration, e2e)
- How CI is configured
- Where docs live and how to navigate them
- Known limitations and next steps
- Troubleshooting section
- Security notes:
 - how you avoid user enumeration
 - how you protect sessions
 - CSRF approach
 - what you do not log

11) ADR Requirements

You must include at least 10 ADRs in `/docs/adr`.

Each ADR must include:

- context
- decision
- options considered
- consequences
- follow-ups / future work

Your ADRs must include decisions for:

- authentication/session approach
 - CSRF approach
 - storage abstraction and file storage strategy
 - schema versioning + migration approach
 - concurrency strategy (ETag/version)
 - API error model (Problem Details)
 - audit logging design
 - import/export design
 - frontend state/data fetching approach
 - test strategy and tooling choices
-

12) Quality Bar

We will verify:

- A fresh clone can run with clear instructions
 - Tests execute cleanly
 - Multi-tenant access control is enforced (no cross-org leakage)
 - Concurrency conflicts are handled and tested
 - File storage is safe and migration works
 - Audit logs exist and are role-gated
 - Docs are structured, navigable, and reflect the system accurately
-

13) Evaluation Rubric

We score across SDLC, not just “features done”.

- Planning and scope management
 - Requirements quality and acceptance criteria depth
 - Design quality (C4, API discipline, security and ops thinking)
 - Implementation quality (structure, correctness, types, validation)
 - Security posture (threat model → mitigations, safe defaults, abuse controls)
 - Testing maturity (unit + integration + e2e, meaningful coverage, CI automation)
 - Maintenance readiness (bugfix/change/security tasks + release notes + post-incident)
 - Handover quality (README, runbook, operational clarity)
-

14) Submission Checklist

Before submitting, confirm:

- Repo includes `/docs` , `/maintenance` , `README.md` , `CHANGELOG.md`
- InMemory and File storage both work; file has `schemaVersion` and migrates v1 → v2
- RBAC works (Member vs OrgAdmin); membership management works
- Audit log is written and viewable by OrgAdmin only
- Import/export works with validation and rejection reporting
- Concurrency conflicts occur correctly and are tested
- Problem Details errors are used consistently with `correlationId`
- CI runs build + lint/format + tests